

Project 1: FIR Filter Design

I. Project Goal

The purpose of this project is to create a functionally correct HLS design for an 11-tap and 128-tap FIR filter to gain an understanding of different ways to optimize HLS. Resource usage and execution time are some metrics used to quantify how effective these optimizations are compared to each other and will be used to find the best design.

II. Questions

Question 1 - FIR11 Baseline: Implement a functionally correct 11-tap FIR filter. Do not apply pragmas or other optimizations. You can take reference from the textbook.

- Report latency, initiation interval (II), in clock cycles.
- Report the number of BRAM, DSP, LUT and FFs used.

$$Throughput (MHz) = \frac{1000}{Clock Period (ns) \times \# Clock Cycles} \quad \text{Equation 1}$$

Q	Modification	Estimated Clock (ns)	Latency (Cycles)	Throughput (MHz)	BRAM	DSP	LUT	FF
1	baseline FIR11	6.912	19	7.6145	0	2	383	733

After running the FIR11 baseline, the table above are the numbers we received. Since initiation interval (II) is not added yet, II = 0. We had an estimated clock speed of 6.912 ns and latency of 19 clock cycles. Using Equation 1, we calculated a throughput of 7.6145 MHz using the estimated clock period and the clock cycles. As for the resource usage, 0 block ram (BRAM), 2 custom data paths (DSP48s), 383 lookup tables (LUT), and 733 flip-flops (FF) were used.

Questions 2-7 refers to FIR128. You should have a functionally correct design before starting these questions.

Question 2 - Variable Bitwidths: You can specify a precise data type for each variable in your design. There are many different data types including floating point, integer, fixed point, all with

varying bitwidths and options. The data type provides a tradeoff between accuracy, resource usage, and performance.

Change the bitwidth of the variables inside the function body (namely `coef_t` and `acc_t`) using the `ap_int<>` datatype.

- Try 3 combinations of variable bitwidths of your choice. Report the latency, II, and resource usage (BRAM, DSP, LUT, and FF) for each design in a table.
- What is the minimum bitwidth of `coef_t` and `acc_t` you can use without losing accuracy (i.e., your results still match the golden output)?

Q	Modification	Estimated Clock (ns)	Latency (Cycles)	Throughput (MHz)	BRAM	DSP	LUT	FF
2	baseline FIR 128	6.912	135	1.07	3	2	305	610
2	bitwidth optimization coef_t = 14 acc_t = 14	6.912	135	1.07	2	2	315	588
2	bitwidth optimization coef_t = 14 acc_t = 7	error	-	-	-	-	-	-
2	bitwidth optimization coef_t = 14 acc_t = 8	6.912	135	1.07	2	2	315	588
2	bitwidth optimization coef_t = 4 acc_t = 8	error	-	-	-	-	-	-
2	bitwidth optimization coef_t = 5 acc_t = 8	6.912	135	1.07	2	2	315	588

The latency, II, and BRAM, DSP, LUT, and FF resource usages for different iterations of variable bitwidth of `coef_t` and `acc_t` are shown above. I first modified `coef_t` and `acc_t` to be `ap_int<14>` and `ap_int<14>` respectively since that was what I saw being mentioned in the book. I just wanted something working initially. To find the minimum bitwidth of `coef_t` and

acc_t without losing accuracy I decreased one of them at a time before I hit an error and make test stopped running. I got errors with acc_t being ap_int<7> and coef_t being ap_int<4>. This means the minimum bitwidth of coef_t and acc_t without losing accuracy is ap_int<5> and ap_int<8> respectively. The golden output matches with these two outputs. I did not notice any major performance improvements from changing bitwidth compared to the baseline since the throughput stayed the same as baseline. Changing bitwidth does reduce resource FF usage slightly from 605 to 588, which sort of makes sense.

Question 3 - Pipelining: Pipelining increases throughput, typically at the cost of additional resources. The initiation interval (II) is an important design parameter that affects performance and resource usage. Hint: In this project, pipelining should be applied to the for loop instead of the entire function.

- Report the latency, II, and resource usage of your baseline FIR128 design.
- Turn off the automatic pipelining by using #pragma HLS pipeline off. Report the latency, II, and resource usage of this design.
- Manually pipeline the design using #pragma HLS pipeline II=<value>. Explicitly set the loop initiation interval (II) starting at 1 and increasing in increments of 1 cycle until the throughput stays unchanged (and/or, reaches the same value of a non-pipelined design). Report the latency, II, and resource usage of each design in a table.
- At some point setting the II to a larger value does not make sense. What is that value in this example?
- Vitis HLS may automatically pipeline a loop without any specific pragma. Based on your observation, what is the default II for a pipelined loop used in this case?

Q	Modification	Estimated Clock (ns)	Latency (Cycles)	Throughput (MHz)	BRAM	DSP	LUT	FF
2	baseline FIR 128	6.912	135	1.07	3	2	305	610
3a	HLS Pipeline Off	6.912	449	0.322	1	2	289	390
3b1	II = 1	6.912	135	1.07	2	2	315	588
3b2	II = 2	6.912	262	0.55	1	2	331	491
3b3	II = 3	6.912	389	0.37	1	2	328	458
3b4	II = 4	6.912	516	0.28	1	2	333	458

The latency, II, and resource usage of the base FIR128 design and for questions 3c, 3d, and 3e are shown above. The results from increasing the HLS pipeline II are shown in 3b1, 3b2, 3b3, and

3b4 are shown above too. Turning off HLS pipelining had drastic consequences. It increased latency to 449 cycles and decreased throughput to 0.322 MHz. At least for my case, setting the II to a value larger than 2 does not make sense since it decreases the throughput. This is because as I increased the II to 2, 3, and 4, the throughput progressively decreased from 1.07 MHz for II=1 to 0.28 MHz for II=4. That is why for future code, I am keeping my II to 1. The default II for a pipelined loop appears to be II = 1 since it has the same throughput and relatively similar resource usage as the baseline and unmodified FIR 128. Resource usage in terms of RAM, LUT, and FF did decrease as the II increased. This might tell us increasing II is purposefully using less resources, which decreases performance without giving us much gain. It seems that this needs to be combined with another optimization method to be more effective.

Question 4 - Removing Conditional Statements: If/else statements and other conditionals can limit the possible parallelism and often require additional resources. Rewriting the code to remove them can make the resulting design more efficient. This is known as code hoisting.

Rewrite the code to remove any conditional statements.

- Report the numbers in a table: Compare the latency, II, and resource usage of the automatically pipelined design with / without conditional statements.
- Report the numbers in a table: Compare the latency, II, and resource usage of the non-pipelined design with / without conditional statements.

Q	Modification	Estimated Clock (ns)	Latency (Cycles)	Throughput (MHz)	BRAM	DSP	LUT	FF
2	baseline FIR 128 (no pipelining with conditional statements)	6.912	135	1.07	3	2	305	610
4a	Auto Pipeline ON with conditional statements II=1	6.912	135	1.07	1	2	326	396
4b	No conditional statements and pipelining. II = 1	6.912	135	1.07	1	2	326	396
4c	4a + No conditional statements. II = 1	6.923	117	1.235	2	0	12	5

After rewriting the code to remove any conditional statements, the latency, II, resource usage of the automatically pipelined design with and without conditional statements and non-pipelined

with and without conditional statements is shown above. It seems that pipelining on and removing conditional statements each by their own slightly improved performance does give similar results to each other as shown in the column above for entries 4a and 4b. However, if I turned auto pipelining on and remove conditional statements, the throughput is increased from 1.07 MHz to 1.235 MHz. The amount of resources used, LUT and FF, is also majorly decreased.

Question 5 - Loop Partitioning: Dividing the loop into two or more separate loops may allow for each of those loops to be executed in parallel (via unrolling), enable loop-level pipelining, or provide other benefits. This may increase performance and resource usage.

- Briefly describe the opportunity for loop partitioning in FIR128. Re-write the code to apply your idea.
- Compare the latency, II, and resource usage of the design with / without loop partitioning.
- Apply loop unrolling to the design with loop partitioning. Report the latency, II, and resource usage of this design.
- What is the relationship between loop unrolling and pipelining? Can they be applied together and benefit the design? Justify your answer with experiments / references to past questions.

Q	Modification	Estimated Clock (ns)	Latency (Cycles)	Throughput (MHz)	BRAM	DSP	LUT	FF
3	FIR128 Baseline	6.912	135	1.07	3	2	610	305
5c	loop unrolling and partitioning	6.912	268	0.5398	2	2	354	380
5d	loop unrolling and pipelining	6.912	268	0.5398	2	2	354	380

The initial script I started with for the FIR128 has one for loop which updates the delay line then does the multiplication accumulate computation. One way we can potentially improve performance is to apply loop partitioning and split the big loop into two separate loops so that they can run in parallel through the process of loop unrolling. In my new design, I will create a loop that only updates the delay line and then the other loop will compute the multiplication and addition. I added `#pragma HLS PIPELINE II=1` after the function “for” as directed by the textbook to pipeline each loop. After applying loop partitioning, the row labelled 5c is the resulting output. It seems that resource usage in terms of BRAM, LUT, and FF decreased, but throughput was cut in half from 1.07 MHz in the baseline to 0.53 MHz. This result tells me that loop unrolling and partitioning cut resource usage in half at the sacrifice of throughput. I believe

that loop unrolling and pipelining can be applied together to benefit the design. I think if I combined this optimization with what I did in Question 4C, it could better take advantage of the parallelization optimization we did in this question. I will explore this in the final section.

Question 6 - Memory Partitioning: The storage of the arrays in memory plays an important role in area and performance. On one hand, you could put an array entirely in one memory (e.g., BRAM). But this limits the number of read and write accesses per cycle. Or you can divide the array into two or more memories to increase the number of ports. Or you could instantiate each variable as a register allowing simultaneous access to all the variables at every clock cycle.

Read the textbook about the memory partitioning parameters: block, cyclic, and complete.

- Explore array partitioning options for both arrays in your design from Question 5. Report the latency, II and resource usage. Which partition gives the best performance?
- Loop unrolling and memory partitioning are often used together. Try disabling loop unrolling or array partitioning. Report the effects.

Q	Modification	Estimated Clock (ns)	Latency (Cycles)	Throughput (MHz)	BRAM	DSP	LUT	FF
2	baseline FIR 128	6.912	135	1.07	3	2	610	305
6a	complete partitioning	6.912	267	0.541	0	2	3436	8667
6b	block partitioning factor = 2	6.912	269	0.538	4	2	502	423
6c	block partitioning factor = 4	6.912	269	0.538	8	2	633	425
6d	cycle partitioning	6.912	268	0.539	2	2	354	380
6e	complete partitioning unroll factor = 2	6.912	139	1.041	0	4	6782	8977
6f	cycle partitioning unroll factor = 2	6.912	204	0.709	2	4	516	626
6g	block partitioning unroll factor = 2	6.912	205	0.709	4	4	817	743

I adapted the code from Question 5 to try to implement memory partitioning optimizations. Comparing the latency, II, and resource usage, it seems that complete partitioning has the best performance as shown in 6e when paired with an unroll factor of 2 since it has a throughput of 1.041 MHz compared to cycle and block partitioning at 6f and 6g respectively who have a throughput of only 0.709 MHz. The resource usage of complete partitioning is high at LUT of 6782 and FF of 8977, which may explain the higher throughput. If we were to take away loop unrolling when commenting out `#pragma HLS unroll factor=2`, the throughput is halved in 6a-6d on the table. Interestingly, for 6a, although complete partitioning with no unrolling has a much higher resource utilization, throughput is relatively similar to the other memory partitioning optimizations, which tells me that complete and loop unrolling must be paired together to work properly. Another observation that I made is that complete partitioning relies mainly on lookup tables and flip flops and does not use any BRAM while block and cycle partitioning does use some BRAM.

Question 7 - Best Design: Combine any number of optimizations to get your best architecture. A design with high throughput will likely take a lot of resources. A design with small resource usage likely will have lower performance, but that could still be good enough depending the application goals.

- Combine any number of optimizations to get your best throughput. What optimizations did you use to obtain this result? Report the latency, II, throughput (in MHz). It is possible to create a design that outputs a result every cycle, i.e., get one sample per cycle, so a throughput of 100 MHz (assuming a 10 ns clock).
- Report the resource usage of your design with the best throughput. Explain why the resource usage is high compared with the baseline in Question 2.

Q	Modification	Estimated Clock (ns)	Latency (Cycles)	Throughput (MHz)	BRAM	DSP	LUT	FF
2	baseline FIR 128	6.912	135	1.07	3	2	610	305
7a	bitwidth, pipelining=1, conditional statement removal, loop and memory partitioning	6.923	30	4.815	0	0	4540	7002
7b	bitwidth, pipelining=4,=1, conditional statement removal,	6.923	30	4.815	0	0	4423	6650

	loop and memory partitioning							
--	------------------------------	--	--	--	--	--	--	--

After reviewing over all my data to see what optimizations led to the best throughput, I combined them all into one script. In my best design after implementing bitwidth optimization to be ap_int<5> and ap_int<8> respectively, adding pipelining with II = 1, removing conditional if statements within the for loop and loop and memory partitioning, I was able to increase the throughput from a baseline value of 1.07 MHz to a whopping 4.815 MHz. I basically was able to increase performance by almost 5 times with the combination of these optimizations. I did also play around with the pipelining and unroll factor options to see if I can squeeze out more performance in my script. I found that pipelining and adding an array partition to the variable c does also help in decreasing the latency as well. If I increased the pipelining II = 4 it uses slightly less LUT and FF compared to if I had pipelining at II = 2. Although my approach uses no BRAM and DSP and heavily relies on FF and LUT, it results in a much higher throughput implementation of the FIR 128 filter. My results for my final experiments are shown above with 7b being the best performing implementation since it has the highest throughput and a slightly lower resource usage than my second best implementation 7a. As shown in the table above, the resource usage of 7a and 7b is much higher than the baseline implementation because I am utilizing II = 4 pipelining at the beginning for the coef_t variable at the beginning, II = 1 for each of the two loops I used, and setting each loop to have an unroll factor of two, the resources are shared sequentially over multiple cycles. Additionally, more flip flops and look up tables are utilized in this implementation, which cuts down on run time but are not as efficient resource-wise as using the BRAM.

Q	Modification	Estimated Clock (ns)	Latency (Cycles)	Throughput (MHz)	BRAM	DSP	LUT	FF
B	baseline FIR 128	6.912	135	1.07	3	2	305	610
2	bitwidth optimization coef_t = 5 acc_t = 8	6.912	135	1.07	2	2	315	588
3b1	II = 1	6.912	135	1.07	2	2	315	588
4c	4a + No conditional statements. II = 1	6.923	117	1.235	2	0	12	5
5d	loop unrolling and pipelining	6.912	268	0.5398	2	2	354	380

6e	complete partitioning unroll factor = 2	6.912	139	1.041	0	4	6782	8977
7a	bitwidth, pipelining, conditional statement removal, loop and memory partitioning	6.23	30	4.815	0	0	4461	6684

The table above is me picking out the best performing FIR128 run from each of the questions. I wanted to separate out the best ones to compare them with each other. The two MATLAB generated plots below compare the throughput and resource usage of the best performing FIR128 optimization implementations. Since throughput was an easier metric to compare than resource usage objectively than resource usage, I used that to single out the best runs from each question.



